

VCD93K User Manual

VCD93K Application Notes
February 1, 2017

Advantest Europe GmbH
Branch / Zweigniederlassung Boeblingen
Herrenberger Str. 130
D-71034 Boeblingen

Contents

1	Introduction	1
1.1	Audience	1
1.2	Concept	1
1.3	Synchronous and Asynchronous signals	2
1.4	Test Program Building Process using VCD93K	3
2	VCD93K Operational Principles	5
2.1	Cyclization of Event Based Data	5
2.2	Understanding Event Based Data	6
2.2.1	VCD Files	6
2.2.2	EVCD Files	6
2.3	VCD93K Cycle Matching	7
2.3.1	Force User-Defined Timing	7
2.3.2	Condition Based Waveform Matching	7
2.3.3	Waveforms Definition	8
3	Using VCD93K	9
3.1	Introduction	9
3.2	Setting Up Signals	9
3.2.1	Import Signals from Event Based Files	10
3.2.2	Updating Signal Properties	10
3.3	Settings Up Groups	13
3.4	Defining Signal Timing	14
3.4.1	VCD93K Waveform Definition	14
3.4.2	VCD93K Forced Waveforms	16
3.4.3	VCD93K Conditional Waveforms	16
3.4.4	Multiport Mode	19
3.5	Settings Up AC Specs	20
3.6	Setting Up the VCD93K Setup File	21
3.6.1	Creating VCD93K Setup Template Files	21
3.7	Setup File Parameters	22
3.7.1	(E)VCD reader Parameters	22
3.7.2	ATE generator parameters	25
3.8	Using Command Line Interface (CLI) of VCD93K	27
3.8.1	Command Line Options Description	27
4	Generating Advantest 93000 (SmarTest 6.X/7.X) Output From STIL	29
4.1	How Do I Generate Advantest 93000 (SmarTest 6.X/7.X) Output from STIL?	29
4.2	Types of Generated Files and File Naming Convention	29
4.3	STIL Elements Usage	30
4.3.1	Signal and SignalGroup Translation	30

4.3.2	Timing Translation	31
4.3.3	Pattern Translation	34
4.3.4	Using Concurrent Test (MultiPort)	35
4.3.5	Using X-modes	36
4.3.6	BreakPoint Statement Translation	36
4.4	Elements Naming	37
4.4.1	SmarTest Name Length Restrictions	38
4.5	Customizing 93000 Files	38
5	Generating Advantest 93000 (SmarTest 8.X) Output From STIL	47
5.1	How Do I Generate Advantest 93000 (SmarTest 8) Output from STIL?	47
5.2	Types of Generated Files and File Naming Convention	47
5.3	STIL Elements Usage	48
5.3.1	Signal and SignalGroup Translation	48
5.3.2	Timing Translation	49
5.3.3	DCLevels Translation	52
5.3.4	Specs Translation	53
5.3.5	Pattern Translation	53
5.3.6	Using MultiPort	55
5.4	Elements Naming	56
5.5	Customizing 93000 Files	56
6	Appendix	61
6.1	What is CSV File ?	61
6.2	How to use CSV File?	61
6.3	How to define pins direction ?	64
6.4	How to define input/output waveforms timing ?	65
6.5	How to use FSDB dump files	65
7	Glossary	67

Chapter 1

Introduction

1.1 Audience

The VCD93K User Manual is intended for test and design engineers of integrated circuits.

1.2 Concept

The purpose of VCD93K software is to convert an event-based test (simulation results which represented in (E)VCD event based formats) using pre defined timing (according to the Device Spec) for the signals that should appear in the final output files, to ATE test program (for various ATE).

The inputs to VCD93K are design simulation files (Verilog VCD/EVCD dump files).

The Output is the Test Program files to run on the Tester.

When designing a device, the simulation test file is event based. The test generated by the simulator contains events of the device, lined up one after other. With ATE (Automatic Test Equipment) it is currently impossible to execute such a test. ATE uses cycles format re synchronizing all signals within a certain period.

1.3 Synchronous and Asynchronous signals

Important note: In order to convert (E)VCD file into ATE patterns with the VCD93K all the signals should be synchronous to the clock period (cyclic)

For example, you can see in the picture below :

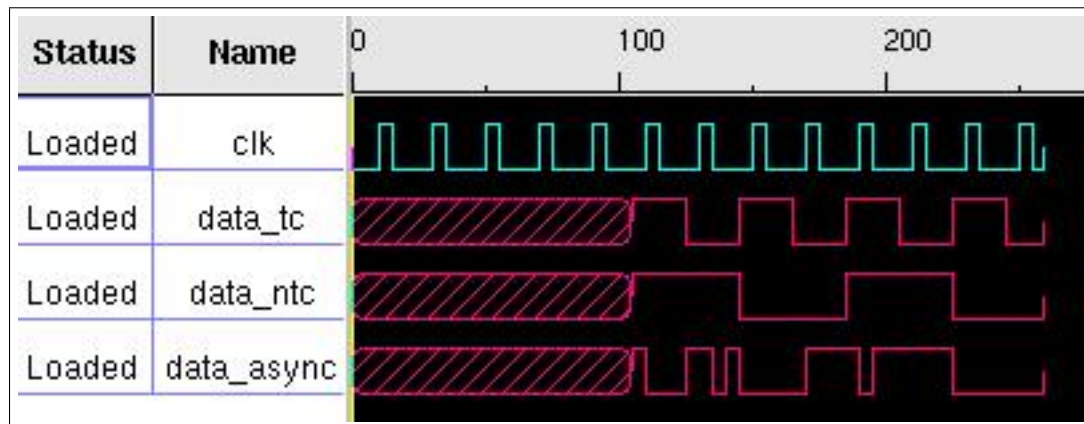


Figure 1.1: signals of VCD file

The two signals data_tc and data_ntc are synchronous and can convert into ATE tester timing. The signal data_async is Asynchronous and can not convert as it into ATE tester timing.

In fact we can convert Async (E)VCD, but the resulting test will differ from the source as we internally sample each signal in a predefined cycle location.

So the resulting test will only drive/compare each signal in a fixed phase while the VCD may have been different.

1.4 Test Program Building Process using VCD93K

For execution of VCD93K it's mandatory to define the following parameters:

1. **Input (E)VCD definition** - the (E)VCD input file
2. **Tester period definition** - cycle unit period for the ATE patterns
3. **Signal direction definition** - direction of each pin according to device spec (For VCD file only).

- (a) Input
- (b) Output
- (c) InOut

Note:

When importing signals from a VCD file, only the names of the pins are extracted. Other signal properties, such as pin direction, are set to defaults (direction = Input) and needed to be modified by the user according to the device spec. By default The direction for EVCD file is inout.

4. **Signal Timing definition** - timing behavioral for each pin according to the device spec
5. **Output target tester definition** - the target tester

Chapter 2

VCD93K Operational Principles

2.1 Cyclization of Event Based Data

Production testers require that all signal waveforms are defined in every cycle during the test. VCD93K needs to read a given event driven formatted test and translate it into a cycle-based ATE format, within which every signal is assigned a pre-defined waveform in every cycle.

The VCD93K methodology offers a flexible approach to cycle matching. It uses the independent definition of the two elements in cycle matching:

- The condition defining *when* (within the event stream) a specific cycle is to be used.
- The cycle definition of *how* the cycle should be implemented using the ATE hardware (which may or may not be similar to the event stream). Part of the cycle definition may include a point where the signal state is sampled from the event stream data.

By separating these two elements, the proposed methodology addresses the issue that simulation does not follow the exact device cycle specification definition. The selection of a specific cycle is done based on the pre-defined condition, which may include any signal in the simulation.

Users can include an artificial indicator of specific device modes to control cycle selection and thus the timing spec selection. The selection is activated on a signal or a group of signals independently. This greatly simplifies the permutations among signals per the different device modes. Section [2.3 VCD93K Cycle Matching](#) describes the specific modes used.

The tool:

- Reads the event driven data.
- Splits the whole sequence into cycle periods pre-defined by the user.
- It then goes through every signal in the list and checks which waveform should be assigned in the current cycle.

- It verifies the conditions, and the first condition matched will assign its corresponding waveform to the signal in the current cycle.
- If no condition is matched in any given cycle, the tool will stop and issue an error.
- When finished assigning a waveform throughout the test, the tool writes the resulting ATE test program and quits.

2.2 Understanding Event Based Data

The event based data is represented by value change dump files (VCD and EVCD formats). Both formats are IEEE standard ones, 1364-1995 and 1364-2001 correspondingly. The data in VCD and EVCD files represents a trace of logic simulation and provides information about the values for each signal at any given time point in the simulation.

2.2.1 VCD Files

There are four states that represent logic values of signals in a VCD file. Signal direction information is not available in VCD. Additional means are used to specify signal direction for VCD93K, for example control pins.

Value	Description
0	Logic low
1	Logic high
Z or z	Tristate
X or x	Unknown

Table 2.2: VCD logic values

2.2.2 EVCD Files

Signal states in EVCD files already include direction information.

Value	Description
Input	
D	Drive zero
U	Drive high
N	Drive unknown
Z	Tri-state
d	Drive zero (two or more drivers active)
u	Drive high (two or more drivers active)
Output	
L	Compare to logic low
H	Compare to logic high
X	Unknown (don't care)
T	Compare to midband
l	Compare to logic low (two or more drivers active)
h	Compare to logic high (two or more drivers active)
Unknown direction	
0	Logic low (both input and output are active with 0 value)
1	Logic high (both input and output are active with 1 value)
?	Unknown
F	Tri-state (input and output disconnected)
A	Unknown (input 0 and output 1)
a	Unknown (input 0 and output X)
B	Unknown (input 1 and output 0)
b	Unknown (input 1 and output X)
C	Unknown (input X and output 0)
c	Unknown (input X and output 1)
f	Unknown (input and output tri-stated)

Table 2.4: EVCD logic values

2.3 VCD93K Cycle Matching

The timing file used by VCD93K for cycle matching consists of the following elements defined for each signal:

- Waveform character.
- List of waveform definitions.
- Condition for each waveform.

2.3.1 Force User-Defined Timing

Specifying true in the Condition column of a timing spreadsheet enables you to assign the associated waveform regardless of actual signal behavior in the simulation.

2.3.2 Condition Based Waveform Matching

Using conditions enables you to assign the associated waveform only when the condition specified is met. The condition consists of signal states at specific timepoints, both inside and outside cycle boundaries.

2.3.3 Waveforms Definition

Each waveform defined in the timing file is a sequence of edges (zero or more) at specific timing points, which describe the desired signal behavior during a given tester cycle. Table 2.5 demonstrates specific VCD93K waveforms. This table is *not* a part of the VCD93K timing file and is provided for explanation only.

VCD93K Waveform	Description
100ns:D	Drive zero at 100ns from the beginning of the tester cycle.
100ns:U	Drive high at 100ns from the beginning of the tester cycle.
100ns:D; 250ns:U	Clock pulse with negative edge at 100ns and positive edge at 250ns.
Tper*0.25:F	Edge F is a shortcut for "drive data". The value is selected from the simulation at Tper*0.25. D is used if the simulation data is 0 and U is used if the simulation data is 1.
Tper*0.75:R	Edge R is a shortcut for "compare data". The value is selected from the simulation at Tper*0.75. Edges L, H, X and T are used when the simulation data is compared with low, high, unknown or midband used respectively.
Tper*0.5:F@(Tper*0.75)	In this waveform, the data to be driven is taken from Tper*75, even though it is applied at Tper*0.5.

Table 2.5: Examples of waveforms

Table 2.6 describes timing edges supported by VCD93K.

Timing Edge	Description
D	Drive zero
U	Drive high
N	Drive unknown
P	Drive previous
Z	Turn driver off
L	Compare to logic low
H	Compare to logic high
X	Compare to unknown
T	Compare to value between logic high and logic low (midband)
F	Drive data (low or high) depending on actual data in the simulation. Signal direction must be input for this edge.
R	Compare data (low, high, unknown or midband) depending on the actual data in the simulation. Signal direction must be output for this edge.
S	Drive or compare data, depending on actual data and signal direction in the simulation.

Table 2.6: STIL timing edges supported by VCD93K

Chapter 3

Using VCD93K

3.1 Introduction

VCD93K operates via a command line interface, controlled by a setup file. VCD and EVCD event based formats are converted into cycle driven format files, according to the settings in the setup file. In this setup file, you define the following main setup parameters and inputs:

- Paths to the following files in:
 - input (optional, can be also specified from command line).
 - timing definition (mandatory).
 - pins definition (mandatory).
 - pin groups definition (optional).
- Cycle width (mandatory).
- List of signals to be included in the output STIL file (optional, by default all signals are included).

3.2 Setting Up Signals

VCD93K converts an event-based simulation file into a cyclized patterntest for a tester.

In the simulation file, a signal's direction is unknown. This means that the test engineer must specify the direction of the pin in VCD93K (in the pins definition file). VCD93K uses this information to decide whether the tester should drive or compare data. There are three types of pin direction: input, output, and input/output (I/O). When a pin direction is input or output, it means that this direction stays the same during the functional/structural test. When a pin direction is I/O it means that this pin is bidirectional; its direction may change during the test. VCD93K needs to know when this pin is input and when it is output.

VCD93K uses an additional internal signal to identify I/O pin direction. This internal signal should be included in the simulation file. This signal is the I/O control pin. You have to define, for each inout pin, its control signal. This is done by entering it in the I/O control pin property in the pins definition file. See [Direction Controls](#) for more details.

3.2.1 Import Signals from Event Based Files

In order to import the existent list of signals from the VCD/EVCD file automatically into the pins definition file, you need to run VCD93K with the following flag:

```
--> -work_mode: gen_pins
```

Run command example:

```
vcd93k -work_mode: gen_pins -setup MySetup.py
```

The pin list is imported to the pins definition file, defined in the setup file in the following parameter:

```
pins_file = 'Pins.csv'
```

The pin list contains only pins that are defined in the imported file. The direction of the pins is set to in if the input file is VCD, and inout in case of EVCD.

3.2.2 Updating Signal Properties

When importing signals from a VCD file, only the names of the pins are extracted. Other signal properties, such as pin direction, are set to defaults (in or inout) and need to be modified by the user.

	A	B	C	D	E	
1	<VS Pins>	1.0				
2	Pin	Type	I/O Control	State Map	SSP	
3	D_dir	in			0	
4	A0	in			0	
5	A1	in			0	
6	CLK0	in			0	
7	CLK1	in			0	
8	CLK2	in			0	
9	CS_	in			0	
10	D0	in			0	
11	D1	in			0	
12	D2	in			0	
13	D3	in			0	
14	D4	in			0	
15	D5	in			0	
16	D6	in			0	
17	D7	in			0	
18	GATE0	in			0	
19	GATE1	in			0	
20	GATE2	in			0	
21	OUT0	in			0	
22	OUT1	in			0	
23	OUT2	in			0	
24	RD_	in			0	
25	WR_	in			0	
26	</VS Pins>					

Figure 3.1: Example of imported pins file

The pins definition file contains the following fields:

- **Pin:** Specifies the Pin name taken from the imported file. Mandatory.
- **Type:** Specifies the pin direction, which can be one of the following: in, out, inout. Mandatory. When importing signals from a (E)VCD file, pin direction is set to default values and should be modified by the user.
- **I/O Control:** Controls the direction of the pin. This field is only valid for VCD inout pins for which it is mandatory.
- **State Map:** Not supported in this product.
- **SSP:** Enables (1) or disables (0, the default) of Smart Strobe Positioning (SSP). The purpose of SSP is to minimize the number of output waveforms and strobe positions created due to output waveform phases. For each signal, VCD93K tries to locate a common strobe position for all output waveforms of that signal.

Direction Controls

Direction control enables you to specify when a VCD inout signal's value should be translated to the input state and when to the output state, based on current values of one or two control signals.

The format of the direction control definition is:
 "in:pin=states,out:pin=states".

For example, consider the below line in pins definition file:

```
1 "D[7]"      "inout"      "in:D_dir=0,out:D_dir=1"
```

It has the following effect:

When D_dir = 0 (Low) signal D[7] direction is input

When D_dir = 1 (Up) signal D[7] direction is output

I.e. when the value of the D_dir signal is 0, all values of D[7] are translated to input states (0 to D, 1 to U, etc.) when the value of D_dir is 1, all values of D[7] are translated to output states (0 to L, 1 to H, etc.).

	A	B	C	D	E
1	<VS Pins>	1.0			
2	Pin	Type	I/O Control	State Map	SSP
3	D0	inout		[D_Dir, D0] 10->D ?X->X 0Z->X 01->H 00->L	0
4	D1	inout	in:D_dir=0,out:D_dir=1		0
5	D2	inout	in:D_dir=0,out:D_dir=1		0
6	D3	inout	in:D_dir=0,out:D_dir=1		0
7	D4	inout	in:D_dir=0,out:D_dir=1		0
8	D5	inout	in:D_dir=0,out:D_dir=1		0
9	D6	inout	in:D_dir=0,out:D_dir=1		0
10	D7	inout	in:D_dir=0,out:D_dir=1		0
11	GATE0	in			0
12	GATE1	in			0
13	GATE2	in			0
14	OUT0	in			0
15	OUT1	out			1
16	OUT2	out			1
17	RD_	in			0
18	WR_	in			0
19	</VS Pins>				

Figure 3.2: Example of pins definition file with direction controls

3.3 Settings Up Groups

Defining the pins into groups is performed by defining the groups in the pin groups file. This file is defined in the setup file in the following parameter:

```
groups_file = 'Groups.csv'
```

The Pins Group file contains the following fields:

- **Group** - Pin Group name (mandatory).
- **Pins** - Group members wildcards (?*) are allowed (mandatory).

The following is an example of a Pin groups file where:

- Group **ADDR** includes [A0, A1, A2, A3] signals
- Group **DBUS** includes [D0, D1, D2, D3, D4, D5, D6, D7] signals

	A	B
1	VS Groups	1.0
2	Group	Pins
3	ADDR	A0,A1,A2,A3
4	DBUS	D0,D1,D2,D3,D4,D5,D6,D7
5	<VS Groups	

Figure 3.3: Example of pin groups file

3.4 Defining Signal Timing

The timing definition for each of the signals is done using a timing file. VCD93K uses the timing file specified in the setup file in the following parameter:

timing_file = 'Timing.csv'

3.4.1 VCD93K Waveform Definition

The timing file contains the following fields:

	A	B	C	D
1	VS Timing	1.0		
2	Pin	WFC	Waveform	Condition
3				
4	<VS Timing			

Figure 3.4: Example of an Empty Timing File

- **Pin** - Pin name (mandatory).
- **WFC** - Waveform character (optional).
- **Waveform** - Waveform description (mandatory).
- **Condition** - Waveform matching condition (mandatory).

Each line defines a waveform for the pin specified in the first column. During STIL generation, VCD93K tries to match pin waveforms, in order of definition. The way of matching a specific waveform depends on its condition. If matching of one waveform fails, VCD93K attempts to apply the next one. If all waveforms defined for the specific pin cannot be matched, an error is generated.

VCD93K supports two types of waveform matching:

- For forced waveforms, the condition field contains the keyword: **'true'**.
These waveforms always match. A separation between input and output cycles is supported by use of **F/R** events.
- Conditional waveforms have the actual condition expression defined. VCD93K checks the condition and applies the waveform if, and only if, the condition is resolved to true.

The WFC field defines an ASCII character to use as a waveform character in STIL timing. If WFC is not specified, VCD93K selects a STIL waveform character automatically, based on waveform events. WFC cannot be defined for waveforms featuring **S/F/R** events (see below).

The Waveform field defines waveform events, separated by a semicolon, e.g., **0ns:D; 10ns:U; 20ns:D**. Each event consists of a time and an action. Time is an expression that can include constants as well as specification parameters and equations.

The following actions are supported:

- D: Driver on, drive low.
- U: Driver on, drive high.
- N: Driver on, drive high.
- Z: Driver off.
- L: Compare low.
- H: Compare high.
- T: Compare midband.
- X: Compare off.
- P: Drive prior (supported only in forced/conditional waveforms).
- S: Use pattern data (supported only in forced/conditional waveforms).
- F: Use pattern data, input only (supported only in forced/conditional waveforms).
- R: Use pattern data, output only (supported only in forced/conditional waveforms).

3.4.2 VCD93K Forced Waveforms

For forced waveforms, the condition field contains the keyword **'true'**. These waveforms are used/assigned by VCD93K unconditionally. VCD93K does not check a condition before assignment. The only exception is the signal direction, which is checked by VCD93K.

Forced waveforms support the special actions **S/F/R**. Instead of defining a specific pin behavior, these actions mean that the (E)VCD state is sampled and substituted in the waveform. **S** actions match every (E)VCD state; **F** actions only match input states; **R** actions only match output states. Therefore, using **F/R** actions enables you to define separate forced waveforms for input and output cycles. In output cycles, a waveform with **F** is not applied. Similarly, for input cycles, a waveform with **R** is skipped.

For **S/F/R** events, in addition to time and action, a sample point can be provided, e.g., **10ns:S(20ns)**. Sample points instruct VCD93K to sample pattern data in the specific point of the current cycle. In this example, VCD state is sampled at 20 ns from the cycle start. By default, if a sample point is not specified, event time is used (10 ns in the example).

Waveforms with **S/F/R** actions can generate multiple STIL waveforms. For example, waveform **0ns:F** can result in the generation of **0ns:D** or **0ns:U** waveforms depending on the VCD sampled data in the resulting STIL. Consequently, WFC cannot be defined for these waveforms.

	A	B	C	D
1	VS Timing	1.0		
2	Pin	WFC	Waveform	Condition
3	OUT		90ns:R	true
4	IO[1]		80ns:R	true
5	IO[1]		20ns:F	true
6	TCK		50ns:F	true
7	<VS Timing			

Figure 3.5: Example of a Timing File with Forced Waveforms

3.4.3 VCD93K Conditional Waveforms

In addition to the separation between input and output cycles, forced waveforms have a mechanism which controls the selection of waveforms for a specific pattern cycle. When such a mechanism is needed, conditional waveforms should be used. They differ from forced ones in having a real conditional expression in the condition column. VCD93K checks the condition, and applies the waveform if, and only if, the condition is resolved to true.

Conditional expressions consist of signals, signal value and time.

Expression	Description
<code>signal == 'states'</code>	Signal equals one of the states
<code>signal != 'states'</code>	Signal is not equal one of the states
<code>signal == 'states' [@check-point]</code>	Signal equals one of the states at specified checkpoint
<code>signal1 == 'states' [@check-point] and signal2 == 'states' [@check-point]</code>	Complex expression that checks that both signal1 and signal2 equals on of the specified states
<code>signal == 'states' [@check-point] or signal == 'states' [@check-point]</code>	Complex expression that checks that either signal1 and signal2 equals on of the specified states
<code>signal == stable(start,length,ignore direction change)</code>	Complex expression that check that the signal is stable in the specified range

Table 3.1: Available Condition Syntax

states:

Valid states' values for the use in condition are: D,U,N,Z,L,H,T,X. You are able to use a list of possible states, separated by pipe sign, i.e. D|U. Wildcards can be useful in bus states; ? means any bit (i.e. D? matches DD, DU etc), * means any sequence of bits.

In addition, hexadecimal values can be used to make bus-checking conditions more compact. Hexadecimal values should have a prefix of 'xi:' (in this case they will be compared to input states) or 'xo:' (for outputs). For example, "BUS == 'xi:F0'@(t0)" is equivalent to "BUS == 'xi:UUUUDDDD'@(t0)".

check-point:

Checkpoint defines the exact location in the simulation flow where the condition is evaluated.

VCD93K checks the values of the specified signal at the specified time point, and compares the result to the value specified by the user.

For example, if the waveform has a condition **D_dir == 'U'@(t0+5ns)**, the waveform is only applied if signal **D_dir** has a state **U** at 5 ns from cycle start (t0 stands for current cycle start).

It is possible to combine multiple checks, using AND/OR logical operations, for example **D_dir == 'U'@(t0) and CLK == 'D'@(t0)**.

Stable:

A special type of condition checks the stability of the signal value in a specified interval. The format of the condition is:

`signal == stable(start,length,ignore direction change)`.

It accepts three parameters:

- Start of the interval
- Length of the interval

- Optional "ignore direction change" flag (default is 0). If the flag is set to 1, signal value changes are ignored as long as they are only direction changes (e.g., input high to output high).

For example, the condition `SIG0==stable(t0+10ns,20ns)` is only true if the signal SIG0 does not change value between 10ns and 30ns from cycle start.

Examples:

- `(S1 == '1') @(t0-5ns) and (S3 == 'X')`
True when S1= '1' 5ns before current cycle start (t0) , and S3 = 'X' at cycle start.
- `(Data == '0010??')@t0`
True when data starts with '0010' at current cycle start (t0). The 2 last bits are ignored, but the overall state size must be 6.

3.4.4 Multiport Mode

VCD93K enables you to split a signal set into a number of 'ports', that is groups of signals which may have a cycle width different from the cycle width of other ports. For each port, a separate STIL Waveform Table and a separate pattern are generated.

The ports file is a csv sheet with the following fields:

- **Port** - Port name (mandatory).
- **Pins** - Port pin list (mandatory).
- **Period** - Port cycle unit (mandatory).

The Port file is defined in the setup file in the following parameter:
ports_file = 'Ports.csv'

The following is an example of a Port file, where:

- Port1 includes [Clk1, D0, D1, D2] signals, cycle unit 30ns
- Port2 includes [Clk2, D3, D4, D5] signals, cycle unit 50ns

	A	B	C
1	VS Ports	1.0	
2	Port	Pins	Period
3	Port 1	Clk1,D0,D1,D2	30ns
4	Port 2	Clk2,D3,D4,D5	50ns
5	<VS Ports		
6			

Figure 3.6: Example of a Port File

3.5 Settings Up AC Specs

Spec variables used in the equations are initialized in an external STIL standard file.

This file is defined in the setup file in the following parameter:

```
specs_file = 'Specs.stil'
```

The following is an example where two specs parameters. **Period** and **sclk_offset** are defined in the timing file and how they are set up in the specs file:

VCD93K imports spec data from the external specs file.

Timing file content example:

	A	B	C	D
1	VS Timing	1.0		
2	Pin	WFC	Waveform	Condition
3	clkin		0*Period+sclk_offset.P;	true
4	<VS Timing			

Figure 3.7: Example of a Timing File with AC Specs Period and sclk_offset

Spec STIL file content example:

```

1  STIL 1.0 {
2    Design 2005;
3  }
4  Spec {
5    Category func {
6      Period      { Typ '64ns';  Min '0ns'; Max '0ns'; }
7      sclk_offset { Typ '1ns';   Min '0ns'; Max '0ns'; }

```

3.6 Setting Up the VCD93K Setup File

3.6.1 Creating VCD93K Setup Template Files

VCD93K enables you to create a setup template for csv files and the main VCD93K setup file with all of the available operations and commands that you can select and edit, in order to run VCD93K.

Using this file enables you to include signals and their attributes, such as type, direction controls, predefined timing waveforms and state maps.

The VCD93K setup file enables you to control a wide range of aspects of the conversion process.

Create a setup template file

In order to generate a CSV template and the main VCD93K setup file, use the `-template` command line parameter:

For example `> vcd93k -template`

When running this command, VCD93K automatically generates all template files in `./templates` directory and silently exits.

Open the files, add and edit options as desired.

For additional information about the available options, see the [VCD93K setup file](#).

Save the edited setup file.

There is no standard file name extension for setup files (although `.setup` and `.py` are common). No special location is required for the file; you can save it in any convenient directory.

Result

You can now use the setup file by specifying it on the command line using the `-setup` option for example.

```
vcdstil -setup ./MySetup.py
```


3.7 Setup File Parameters

This section describes all VCD93K setup file parameters and commands.

3.7.1 (E)VCD reader Parameters

External Files:

- **pins_file**
Example: `pins_file = 'Pins_direction_setup.csv'`
Specifies the pins direction definitions file.
The parameter is mandatory.
- **timing_file**
Example: `timing_file = 'mydir/mytiming.csv'`
Specifies the signals timing definition file.
The parameter is mandatory.
- **groups_file**
Example: `groups_file = 'mydir/mygroups.csv'`
Specifies the signals groups definitions.
The parameter is optional.
- **ports_file**
Example: `report_file = 'mydir/ports.csv'`
Specifies the ports definitions.
The parameter is optional.
- **specs_file**
Example: `specs_file = './mydir/specs.stil'`
Specifies the specs definitions file in STIL format.
The parameter is optional.

Conversion Attributes:

- **input_files**
Example: `input_files = [ 'f1.evcd', 'f2.evcd' ]`
Specifies the input vcd/evcd files list. The parameter is mandatory.
- **start_time**
Example: `start_time = '1000ns'`
Extraction start point. Useful when part of the (E)VCD should be ignored. Optional. Default is 0.
- **end_time**
Example: `end_time = '2000ns'`

Extraction end point. Useful when part of the (E)VCD should be ignored. Optional. Default is the last time point of (E)VCD file.

- **cycle_width**

Example: `cycle_width = '20ns'`

Cycle width. Mandatory.

- **active_pins**

Example: `active_pins = ['A','B','C']`

List of pins to be included in the output stil file. The parameter is optional (default = select all).

Pin Names Extraction:

- **keep_design_hierarchy**

Example: `keep_design_hierarchy = 1`

If set to 1, the full hierarchical path of a signal is used as the signal name that is, signals will have names like `".mymodule1.mymodule2.mysig"`. If set to 0, only the signal name is stored (i.e., the signal will be named `"mysig"`). This parameter is optional. (default is 1)

- **select_design_modules = []**

Example: `select_design_modules = ['module2','module4']`

The modules in which to process pins. This parameter is optional. (default = select all)

- **keep_design_hierarchy_for_pins = []**

Example: `keep_design_hierarchy_for_pins = ['A','C','D']`

Keep full design path only for a limited set of pins, even if `keep_design_hierarchy` is set to 0. This parameter is optional. (default = [])

Global:

- **vcd_bus_order**

Example: `vcd_bus_order = 'ascending'` This is the default order that the tool assigns to the (E)VCD buses, either `'ascending'` [0:n-1] or `'descending'` [n-1:0].

This parameter is optional. (default = ascending)

- **max_memory_allocation** Example: `max_memory_allocation = 1024`

The maximum memory allocation (in megabytes) to be used by the tool. (default = 1024)

- **max_spike_width** Example: `max_spike_width = '2ns'` The maximum spike width to be ignored by the tool. (default = 0ns)

- **time_unit** Example: time_unit = 'ps'
The default time unit used for equations [fs | ps | ns | us | ms | s] (default = ps)
- **stop_on_fail**
Example: stop_on_fail= 1
Stop if cycle matching fails.
(default = 1, stop after the first time the cycle matching fails)
- **interim_vcd**
Example: interim_vcd = 'try.vcd'
interim vcd file path (interim vcd is not written if the path is empty)
Creates the vcd file with debug information.

3.7.2 ATE generator parameters

- **target**
Select desired ATE Platform. This option duplicates **-target** command line option.
target = 'name_of_target'
Example: target = '93000DD'
- **workdir**
Specifies output directory. This option duplicates **-workdir** command line option.
By default, working directory is './output'.
workdir = 'directory_name'
Example: workdir = './output'
- **project**
Specifies base name for ATE test program files. Default is 'project'.
project = 'project_name'
Example: project = 'project'
- **rename_signals**
Convert names between simulation (E)VCD file and target ATE pins. Signal groups are not allowed.
rename_signals = { 'signal_name1_before' : 'signal_name1_after', 'signal_name2_before' : 'signal_name2_after' }
Example: rename_signals = { 'A[0]' : 'A0', 'A[1]' : 'A1' }
- **sequence_map**
Define pattern states mapping for specific signals. Valid values are 0, 1, L, H, T and X.
sequence_map = { 'signal_name' : { 'current_sequence' : 'new_sequence', ... } , ... }
Example: sequence_map = { 'D1' : { '010' : '111' } }
- **equation_based_timing**
Equation Based Timing. Introduce equation-based timing for STIL files without Spec definition. For example, equation specified as 10ns (when WaveformTable period is 100ns) will become 0.1*per_100, where per_100 is a new Spec variable. Default is 1.
equation_based_timing = value
Example: equation_based_timing = 1
- **Virtual_signals**
The user can add in the final test program signals that were not part of the original (E)VCD file using virtual flag.
Attributes:
 - name: virtual signal name (mandatory, must be unique)
 - type: virtual signal type (optional, one of [In | Out | InOut])

- opcode: virtual signal operator (mandatory)
 - * 'Fixed': Fixed state signal (arg1 = state [DUZLHXT], arg2 = time)
 - * 'Copy': Copy reference signal (arg1 = ref. signal name)
 - * 'Not': Copy reference signal, inverting states (arg1 = ref. signal name)
- args: operator attributes (depend on 'op' selected)

Example:

```
virtual_signals = [ { 'name': "sig_fixed", 'type': "In", 'opcode': "Fixed", 'args':  
[ 'U', '1ns' ] }, { 'name': "clk_copy", 'opcode': "Copy", 'args': [ 'clk' ] }, { 'name':  
"clk_not", 'opcode': "Not", 'args': [ 'clk' ] }, { 'name': "data_not", 'opcode':  
"Not", 'args': [ 'data' ] } ]
```

3.8 Using Command Line Interface (CLI) of VCD93K

The VCD93K can be controlled by command line parameters, setup file parameters or a combination of both. The command line parameters have higher priority than setup file parameters, meaning the latter may be overridden using command line parameters.

3.8.1 Command Line Options Description

Synopsis:

```
vcd93k [-help] [-version] [-timestamp] [-licwait] [-progress] [-template] [-setup setup-file]
      [-work_mode:<str>] [-target:<str>] [-workdir:<str>] [(E)VCD-input-files]
```

The following are the available command line options for VCD93K:

-help	Display help message and exit.
-version	Show version number and exit.
-timestamp	Enable log timestamp.
-licwait	Wait for the license, if not available. The application will wait indefinitely until a license becomes available.
-progress	Show a progress bar.
-template	Generate setup template files under ./templates/directory and exit.
-setup	Specifies the path to the setup file.
-work_mode	[gen_pins] Allows to run in pins extraction mode.
-target	Specifies target ATE name.
-workdir	Specifies output directory.
(E)VCD input-file	Specifies input (E)VCD file, only for export pins to CSV file.

Chapter 4

Generating Advantest 93000 (SmarTest 6.X/7.X) Output From STIL

4.1 How Do I Generate Advantest 93000 (SmarTest 6.X/7.X) Output from STIL?

To generate 93000DD output:

```
> ategen -target 93000DD mysource.stil
```

To generate 93000SD output:

```
> ategen -target 93000SD mysource.stil
```

To generate 93000SS output:

```
> ategen -target 93000SS mysource.stil
```

For more information see ?? in Section ?? and ?? in Section ??.

4.2 Types of Generated Files and File Naming Convention

The table below shows 93000 files generated by VCD93K with a short description. Output files are created in working directory (further denoted as *[workdir]*, see Section ??) and use project directory (further denoted as *[project]*, see Section ??) in their names.

File Type	File Name
<i>[workdir]</i> /configuration/ <i>[project]</i> .cha	pin, group, port and channel configuration file
<i>[workdir]</i> /timing/ <i>[project]</i> .tim	timing file (wavetables, equations and AC specs) May also be generated as three separate files (depends on parameter, see below).
<i>[workdir]</i> /timing/ <i>[project]</i> .wvt, <i>[workdir]</i> /timing/ <i>[project]</i> .eqn and <i>[workdir]</i> /timing/ <i>[project]</i> .ac_spec	separate wavetables, equations and AC specs files
<i>[workdir]</i> /vectors/ <i>[project]</i> .pmfl	pattern master file
<i>[workdir]</i> /vectors/patterns/*.binl	pattern files
<i>[workdir]</i> /vectors/patterns/*.burst	pattern burst definitions
<i>[workdir]</i> /levels/ <i>[project]</i> _dc.eqn	DC equations
<i>[workdir]</i> /levels/ <i>[project]</i> _dc.spec	DC specs

<code>[workdir]/levels/[project].lvl</code>	level master file
<code>[workdir]/data_set/[project].data_set</code>	dataset file - master file to load other files into SmarTest
<code>[workdir]/testflow/[project].tf</code>	Testflow and testsuite template. Contains one testsuite with generated timing, levels and labels

Table 4.1: Advantest 93000 File Naming

4.3 STIL Elements Usage

4.3.1 Signal and SignalGroup Translation

The information from STIL Signals and SignalGroup blocks is used to create the 93000 configuration file.

The STIL signal name is used as the pin name in the target 93000 program. STIL signal direction (In, Out or InOut) is used by VCD93K to define the pin direction.

STIL Signals Block Example

```
Signals {
    "jtag_tdi" In;
    "jtag_tms" In;
    "jtag_tck" In;
    "jtag_trst_" In;
    "jtag_tdo" Out;
}
```

Corresponding 93000 Pin Definitions

```
hp93000,config,0.1
DFPN 10301,"",(jtag_tck)
DFPN 10302,"",(jtag_tdi)
DFPN 10303,"",(jtag_tdo)
DFPN 10304,"",(jtag_tms)
DFPN 10305,"",(jtag_trst_)

CONF I,F160,(jtag_tdi,jtag_tms,jtag_tck,jtag_trst_)
CONF 0,F160,(jtag_tdo)
```

Active STIL SignalGroup domains (selected in the PatternExec block) defines 93000 groups.

STIL SignalGroups Block Example

```
SignalGroups {
    "clock_s" = '"clk[1]" + "clk[2]"';
    "in_s" = '"jtag_tdi" + "jtag_tms" + "jtag_tck" + "jtag_trst_"';
}
```

Corresponding 93000 Group Definitions

```
DFGP I,(clk[1],clk[0]),(clock_s)
DFGP I,(jtag_tdi,jtag_tms,jtag_tck,jtag_trst_),(in_s)
```

The STIL Environment block can be used to define channel allocation. The NameMaps that define channels must start with 93000Channel followed by semicolon and configuration name (e.g. 93000Channel:Standard). Multiple NameMaps are allowed to define multiple configuration files, with the second part of the NameMaps name to serve as a file name (Standard.cha in the example above).

STIL Environment Block Example

```
Environment "Default" {
  NameMaps "93000Channel:Standard" {
    Signals {
      "clk[1]" "11025,Pin Pad25";
      "clk[2]" "12010,Pin Pad10";
    }
  }
}
```

93000 Channel Definitions

```
DFPN 11025,"Pad25",(clk[1])
DFPN 12010,"Pad10",(clk[2])
```

4.3.2 Timing Translation

The STIL Timing block selected in the active PatternExec block is used to generate 93000 timing. Multiple WaveformTable blocks are merged into one 93000 wavetable, but they must share the same Period. Each STIL Waveform is translated into a 93000 waveform using the physical waveform syntax. Waveform numbers and edge allocation are performed automatically.

VCD93K creates a .tim file that either contains wavetables, equation sets and AC specs or contains the command to load this information from the separate files (see use_eqfs settings file parameter).

STIL Timing Block Example

```
Timing {
  WaveformTable t1 {
    Period '100ns';
    Waveforms {
      clk { P { '0ns' D; '25ns' U; '75ns' D; } }
      data { 01 { '50ns' D/U; } }
    }
  }
}
```

Output 93000 tim file

```

WAVETBL "t1"

PINS clk
0 "d1:0 d2:F1N d3:F0N" P

PINS data
0 "d1:0" 0
1 "d1:1" 1

...

EQNSET 1 "t1"

...

TIMINGSET 1 "t1"
period=100.0

PINS clk
d1 = 0.0
d2 = 25.0
d3 = 75.0

PINS data
d1 = 50.0

```

Processing of Multi-bit Waveforms

STIL multi-bit waveforms are fully expanded in 93000 timing. Every resulting waveform is assigned a 93000 PWI (Physical Waveform Index), according to the order of appearance. For example, the following multi-bit waveform:

```
tT { '0ns' L/H[0]; '2.5ns' L/H[1]; }
```

will be expanded to:

```

tt {'0ns' L; '2.5ns' L; }
tT {'0ns' L; '2.5ns' H; }
Tt {'0ns' H; '2.5ns' L; }
TT {'0ns' H; '2.5ns' H; }

```

Consequently, the following 93000 waveforms will be generated:

```

0 "r1:L r2:H" tT
1 "r1:H r2:L" Tt
2 "r1:L r2:L" tt
3 "r1:H r2:H" TT

```

93000 Wavetable Naming

For non-multiport setups each wavetable is assigned a name based on all STIL WFTs that are merged into this wavetable. For multiport setups, a wavetable is created for each port and named as above with an addition of the port name. All wavetable names are trimmed to fit SmarTest maximum wavetable name limit.

Device Cycle Naming

If resource tags exist and are enabled, then they define device cycle names. To enable tags, set use_restag setup parameter to 1. In case the resource tags don't exist, device cycles are named based on STIL waveform characters. If multiple WaveformTables in the STIL file have the same WFC for a given signal in a single Pattern, an index is added to the WFC in order to achieve a unique 93K Device cycle name.

dcd = <resource_tag>

Timing Events Mapping

STIL states are translated to 93000 edges using the following table:

STIL to 93000 Events Map

Event	STIL	93000
Logic Low	A	F0N
Logic High	B	F1N
ForceDown	D	F00
ForceUp	U	F10
ForceOff	Z	FNZ
ForcePrior	P	FNO
ForceUnknown	N	FNO
CompareLow	L	E0
CompareHigh	H	E1
CompareValid	V	EX
CompareOff	T	EI
CompareLowWindow	l	W0
CompareHighWindow	h	W1
CompareOffWindow	t	WI
CompareValidWindow	v	WX
CompareUnknown	X, x	WC, WX, EX
CompareUnknown	X, x	D, C
LogicZ	F	FNZ
ExpectLow	R	EX
ExpectHigh	G	EX
ExpectOff	Q	N

Edge optimization combines a sequence of edges occurring at the same time into a single edge. Currently the following edge combinations are optimized:

Events	STIL	93000
ForceOff,Logic Low	Z,A	F0Z
ForceOff,Logic High	Z,B	F1Z
ForceOff,ForceDown	Z,D	F00
ForceOff,ForceUp	Z,U	F10
Logic Low,ForceDown	A,D	F00
Logic High,ForceUp	B,U	F10
ForceDown,ForceOff	D,Z	F0Z
ForceUp,ForceOff	U,Z	F1Z
Logic Low,ForceOff	A,Z	F0Z
Logic High,ForceOff	B,Z	F1Z

ForcePrior,ForceOff	P,Z	FNZ
ForceUnknown,ForceOff	N,Z	FNZ

VCD93K also performs the following optimizations on the waveforms:

1. Reduce the tristate edges.
2. Remove duplicate states.

4.3.3 Pattern Translation

Pattern blocks in STIL have a one-to-one relationship to 93000 pattern files (BINL files). Exceptions to this rule occur when a pattern is used across several 93000 ports. A pattern master file (PMFL) is always generated as well as a pattern burst (BURST) file. The pattern master file is located in the "../vectors" directory and the pattern burst and all pattern files are located in the "../vectors/patterns" directory.

93000 pattern files consist of the following sections:

1. Pattern Header
2. Comments
3. Pin Data

Each 93000 pattern defines a vector label based on the STIL Pattern block domain name. Repeating signals, as may be found in a STIL Loop pattern statement, are translated to repeat vectors according to optimal usage of the target 93000 model. VCD93K is capable of calculating the optimal number of repeating signals that would warrant the usage of the 93000 repeat statements. If desired this may be configured through the **repeat_break** settings file parameter.

93000 vectors may be compressed in a combination of two ways:

1. Compress data on a per pin basis (ALDC algorithm)
[use_vecc settings parameter]
2. Compress complete pattern file (GZIP algorithm)
[gzip_patterns settings parameter]

VCD93K supports both compression technologies and may be controlled through the settings file.

The following example shows a 93000 pattern as generated by VCD93K.

```
hp93000,vector,0.1
DMAS MTST,VM,0,(@)
DMAS PARA,VM,20,(@)
DMAS SQPG,SM,5,(@)
STML 1
SQLB "label",MAIN,0,4,"pattern1",(@)
SQPG 0,STVA,0,,VM,(@)
SQPG 1,STSA,,VM,(@)
SQPG 2,GENV,10,,VM,(@)
SQPG 3,RPTV,1,10,VM,(@)
SQPG 4,STOP,,,(@)
BCMT 0,1,(@),#90000000006/binary comment/

VECD PARA,VM,0,28,(pin1),#9000000028/binary data/
VECD PARA,VM,0,28,(pin2),#9000000028/binary data/
```

Processing Multi-bit Waveforms

Every time a multi-bit waveform group is encountered in the vector, appropriate PWI is used in the pattern. Following is an example:

```
Signals {
  data InOut;
}

SignalGroups {
  Data = 'data' { DataBitCount 2; }
}

VPattern P1 {
  ...
  V { Data = tt; }
  V { Data = TT; }
  V { Data = Tt; }
  V { Data = tT; }
  ...
}
```

The preceding example will produce the following data sequence in VECD/VECC command: 0321.

4.3.4 Using Concurrent Test (MultiPort)

CCT or concurrent test, refers to the capacity of the 93000 tester to run in parallel different groups of pins (called ports). MultiPort makes the test programming easier and more DUT oriented. Using MultiPort it is possible to program each port separately, just like the DUT ports that may operate independently of each other.

VCD93K supports the MultiPort capability as follows:

1. Using NameMaps definition in the STIL file

```
<signal name> "<port name>,<sync group>,<master clock>,<x factor>,<hx>,<specs scope>,<keep spec variables>"
```

- (a) port name - Defines port name.
- (b) sync group - Defines synchronization group. Relevant only if "generate_clksets" setup parameter is set to 1.
- (c) master clock - Defines master clock.
- (d) x factor - Defines x mode factor. Overrides setup parameter "xmode_factor".
- (e) hx - Defines high speed channels handling. Can be left empty or set to "hx".
- (f) specs scope - Can be set to "global" or "local", to create global (for all ports) or local (per port) specs table.
- (g) keep spec variables - Can be set to 0 or 1. If set to 1, unused spec variables will be kept; if set to 0, dropped. Overrides setup parameter "keep_spec_variables".

Example: Sig1 "Clock,SyncGroup1,1,1,,global,1";

2. Using a ParallelPatList STIL construct in a PatternBurst. When a ParallelPatList is used in a PatternBurst, the VCD93K will automatically create a MultiPort setup.

When using the explicit port definition using NameMaps, the user defines the port name. When using ParallelPatList, port names are created automatically using the pattern port1, port2, etc. based on the order of Patterns, defined in the ParallelPatList object.

4.3.5 Using X-modes

X-modes is a way to compress vectors using the 93000 flexible device cycle definition. Using X-modes, two or more vectors are packed into one vector, while the timing definition is updated to reflect vector compression. Each compressed device cycle will contain an Xfactor number of original cycles.

There are two ways to use X-mode compression feature. In order to compress all vectors in a STIL file please set the `xmode_factor` variable to 2 or higher in the setup file:

```
xmode_factor = 2
```

In case of a multiport setup, the X-mode factor will be applied to all ports. In order to apply different X-Mode factors for each port in a multiport setup, additional parameters are required in ports NameMaps definition of the STIL file:

```
<signal name> "<port name>,<sync group>,<master clock>,<x factor>,  
             <hx>,<specs scope>,<keep spec variables>"
```

Please note that timing will only be created for the actual combinations of vectors in the STIL file.

The X-Mode compression may fail when tester resources are not sufficient (too many edges or waveforms).

4.3.6 BreakPoint Statement Translation

The source STIL file may have a BreakPoint construct defined as part of a Pattern. This BreakPoint statement is used to define Advantest 93000 break waveforms. Break waveform is executed by the tester when it is performing its own tasks (e.g. reloading internal data) and not executing user patterns.

The number of vectors in a BreakPoint statement must be 1 or match the selected X-Mode if X-Modes are used. X-Modes are a process of combining several pattern vectors into one (to save tester memory)). For example:

No X-Modes (X-Mode = 1) - single vector in BreakPoint statement

```
BreakPoint {  
    V { CLK = P; }  
}
```

X-Modes used (X-Mode = 3) - 3 vectors in BreakPoint statement

```
BreakPoint {  
    V { CLK = 0; }  
    V { CLK = P; }  
    V { CLK = 0; }  
}
```

Those three vectors combined will represent a break waveform for signal CLK.
The following example shows the use of the BreakPoint in STIL pattern and its representation in 93000 timing.

STIL BreakPoint Example

```
Timing {
  WaveformTable T1 {
    Period '100ns';
    Waveforms {
      clk { P { '0ns' D; '25ns' U; '75ns' D; } }
    }
  }
}

Pattern Clocks {
  W T1;
  BreakPoint {
    V { clk = 0; }
  }
}
```

93000 Break Waveform

```
WAVETBL "T1"
PINS clk
0 "d1:0 d2:F1N d3:F0N" P
brk "d1:0 d2:F1N d3:F0N"
...
EQNSET 1 "T1"
PINS clk
d1 = 0.0
d2 = 25.0
d3 = 75.0
```

4.4 Elements Naming

Element name	93000 elements
timing wavetable (No MultiPort)	[stil_wft_1]_[stil_wft_2]...[stil_wft_n]
timing wavetable (MultiPort)	[stil_wft_1]_[stil_wft_2]...[stil_wft_n]_[port]
timing equationset	same as wavetable
timing specset	[stil_category]([stil_selector]) or Default if no specs defined
timing timingset	same as wavetable
levels levelset	[stil_level_name]
levels equationset	[stil_level_name]
levels specset	[stil_category]([stil_selector])
test_suite_name	[stil_pattern]
pattern burst seq. label	[stil_pattern_burst]
vector seq. label	[stil_pattern]

Table 4.2: Elements Naming

4.4.1 SmarTest Name Length Restrictions

Because of SmarTest name length restrictions, the names created by VCD93K are subject to postprocessing in order not to exceed the restrictions. The names are trimmed and an automatically generated suffix is added to create a unique name.

The following names are subject to the above processing:

SmarTest name	length restriction
Vector file names and labels	128 characters for Smartest v6.1.1 and higher, 40 otherwise
Wavetable and eqnset names	40 characters
Timingset names	60 characters
Device cycle names	6 characters
Burst names	40 characters
Port names	16 characters
Testsuite names	40 characters

Table 4.3: SmarTest Name Length Restrictions

4.5 Customizing 93000 Files

The customization of the 93000 files is done using the VCD93K configuration file. Following is an example of a configuration file for the 93000DD:

```

1  # Use absolute paths
2  dd_absolute_paths = 1
3
4  # Device cycle naming customization.
5  # The keys are <WFT>_<WFC> strings,
6  # the values are user-defined strings.
7  dd_dcdf_remapping = { 'default_WFT_1' : 'U',
8                        'default_WFT_0' : 'D' }
9
10 # Device cycle naming for specific signals
11 # The keys are signal names, the values are
12 # tables with <WFT>_<WFC> strings as keys, mapped
13 # to user-defined strings
14 dd_dcdf_signal_remapping = { 'clock' : { 'T1_H' : 'h', 'T1_L': 'l' },
15                               'data'  : { 'T1_H' : 'h', 'T1_L': 'l' } }
16
17 # Wavetable naming customization.
18 # The keys are <WFT1>_<WFT2>... strings for regular setups,
19 # and <WFT1>_<WFT2>...<PORT> strings for multiport setups,
20 # the values are user-defined strings.
21 dd_wavetbl_remapping = { 'T1_T2' : 'T_12' }
22
23 # Compress patterns with GZip.
24 dd_gzip_patterns = 1
25
26 # Disable comments in timing files.
27 dd_tim_comments = 0
28
29 # Use EQFS commands in timing and level files.
30 dd_use_eqfs = 1
31
32 # Apply X-Mode factor to all patterns
33 dd_xmode_factor = 2
34
35 # Create all possible X-Mode combinations in timing files
36 dd_xmode_all_cmb = 1

```

The table below describes the complete set of 93000DD package options.

Unless said otherwise, each of the options can also be used with the 93000SD and 93000SS packages. For 93000SD, the option name prefix should be changed from `dd_` to `sd_`. e.g. SmarTest version number option will be called `sd_smar_test_version`. Similarly, for 93000SS, the option name prefix should be changed from `dd_` to `ss_`.

- **`dd_absolute_paths = boolean`**
Enables or disables the use of absolute paths in EQFS commands.
The default is False.
- **`dd_allow_default_levels = boolean`**
Enable or disable default values for DC levels, if not all levels are specified.
The default is True.
- **`dd_allow_nonaligned_match_loops = boolean`**
Allow match loops with number of vectors not aligned to x-mode.
The default is False.
- **`dd_auto_create_signal_groups = boolean`**
Create all inputs and all outputs signal groups for debug purposes.
The default is True.
- **`dd_combinations_allow_new_waveforms = boolean`**
Allow addition of new waveforms in combinations file.
The default is True.

- **dd_combinations_file** = *string*
Turns on waveform incremental mode when set to a file path. The combinations file allows generation of multiple STIL patterns, while keeping consistent timing, compatible with all generated binary patterns. This mode requires identical timing to be shared between all STIL patterns. It is most useful in case of X-Modes, when different STIL patterns have different combinations of STIL waveforms. Waveform incremental mode is compatible with multiport.
The default is None.
- **dd_comment_max_size** = *integer*
Data size for a single BCMT vector comment command.
The default is 640000.
- **dd_create_min_max_typ_tests** = *boolean*
Creates a separate SPECSET for Typ, Min and Max AC and DC spec values. This allows to run the same test with Typ, Min or Max selectors.
The default is False.
- **dd_create_statemapping** = *boolean*
Enable or disable timing files with Statemapping information.
The default is False.
- **dd_dcap_vector_variable_rename** = *dictionary*
Rename default vector variable in case of digital capture. The keys are default variable names, generated by VCD93K (usually a combination of DCap-Filter block name and Pattern block name), the values are user-defined names. 93000DD/93000SS targets only..
The default is {}.

Example:

```
1 dd_dcap_vector_variable_rename = { 'DF1_P1' : 'MyVar1', 'DF1_P2' : 'MyVar2' }
```

- **dd_dcdf_remapping** = *dictionary*
Allows you to customize device cycle names by means of user-supplied table. The keys are <WFT>_<WFC>strings, the values are user-defined device cycles..
The default is {}.

Example:

```
1 dd_dcdf_remapping = { 'SHIFT_L' : 'L', 'SHIFT_H' : 'H' }
```

- **dd_dcdf_signal_remapping** = *dictionary*
Allows you to customize device cycle names (on a signal basis) by means of user-supplied table. This keys are signal names, the values are dictionaries, formatted as dd_dcdf_remapping..
The default is {}.

Example:

```
1 dd_dcdf_signal_remapping = { 'SCANOUT0' : { 'SHIFT_L' : 'L', 'SHIFT_H' : 'H' },
2                               'SCANOUT1' : { 'SHIFT_L' : 'P', 'SHIFT_H' : 'C' } }
```

- **dd_default_levels_summary** = *boolean*
Print information about signals using default levels.
The default is False.

- **dd_digital_capture_mode** = *string*
Choose SmarTest capture mode between 'std_selective' or 'HRPF'. 93000DD/93000SS targets only..
The default is 'std_selective'.
- **dd_digital_capture_res** = *string*
Choose smartest capture resolution between 'std' or 'double'. 93000DD/93000SS targets only..
The default is 'std'.
- **dd_error_on_long_comments** = *boolean*
Error out on comments violating SmarTest limit (depends on SmarTest version).
The default is False.
- **dd_expand_loops** = *boolean*
Expand original loops in source STIL files.
The default is False.
- **dd_full_incremental_log** = *boolean*
Enable or disable printing full incremental mode report.
The default is False.
- **dd_generate_ascii_pattern** = *boolean*
Generate ascii (.avc) pattern.
The default is False.
- **dd_generate_clksets** = *boolean*
Support multiple clock domains.
The default is False.
- **dd_generate_hvm** = *boolean*
HVM support. Triggers generation of ScanInstanceAttachment type of VCAT pattern command. 93000DD/93000SS targets only..
The default is False.
- **dd_generate_vmap** = *boolean*
Enable or disable scan diagnostics.
The default is False.
- **dd_gzip_pattern** = *boolean*
Enable or disable GZip compression of vector files.
The default is False.
- **dd_hvm_cycle_info** = *boolean*
Include vector and cycle information in ScanInstanceAttachment (VCAT) pattern command (scan_test_name field). Requires dd_generate_hvm flag to be turned on. 93000DD/93000SS targets only..
The default is False.
- **dd_keep_all_specsets** = *boolean*
Keep unused specsets.
The default is False.
- **dd_keep_spec_variables** = *boolean*
Keep unused AC and DC variables.
The default is False.

- **dd_keep_wfc_order** = *boolean*
Enable or disable WFC ordering as in source STIL file.
The default is False.
- **dd_levels_group_exclude_pins** = *array*
Exclude pins from grouping in level files.
The default is [].
- **dd_optimize_receive_edges, dd_optimize_drive_edges** = *boolean*
Enable or disable edge optimization. For receive edges, compare don't care and window close edges are removed, if not needed or duplicated. For drive edges, repeated drives of the same values are removed and edges are downgraded in order not to require tristate capable edges..
The default is True.

Examples of Edge Optimizations

<i>Regular Waveform</i>	<i>Optimized Waveform</i>
d1:0 d2:0	d1:0
d1:0 d2:1	d1:0 d2:F1N
d1:Z r1:X r2:X	d1:Z r1:X
d1:Z d2:Z r1:W0 r2:WX r3:X	d1:Z r1:W0 r2:WX

- **dd_pad_xmode_vectors** = *boolean*
Pad last x-mode vector .
The default is False.
- **dd_pattern_memory_type** = *string*
Define pattern memory type:
VM (for SmarTest versions until 7.0.0),
SM (for all SmarTest versions),
SHMEM (for SmarTest versions 7.0.1 and higher),
auto..
The default is auto.
- **dd_pin_based_tim** = *boolean*
Generate timing based on pin names and without the use of pin groups if selected..
The default is False.
- **dd_repeat_break** = *integer*
If left at default value (None), loops will be flattened if $(\text{loop_size} * \text{loop_count} + 6) / 7 < 1 + (\text{loop_count} + 6) / 7$. If set to an integer (not default), all loops with total number of vectors smaller than the specified number will be flattened..
The default is None.
- **dd_repeat_compression** = *boolean*
Compress identical vectors to single vector loops.
The default is False.
- **dd_repeat_compression_ignore_comments** = *boolean*
When repeat compression is on: compress identical vectors to single vector loops although they don't have the same comment. The different comments from the vectors will be placed before the created loop..
The default is True.
- **dd_scan_multiport** = *boolean*
Automatically create separate ports for scan and non-scan signals.
The default is False.

- **dd_stable_multiport** = *boolean*
Automatically create separate ports for stable and non-stable signals.
The default is False.
- **dd_skip_unused_signal_groups** = *boolean*
Don't create signal group definitions if those groups are not used in timing and level declarations in STIL.
The default is False.
- **dd_smarTest_version** = *string*
Controls output behavior depending on SmarTest version.
The default is '6.0.0'.
- **dd_statemap_selector** = *string*
Statemapping selector name. If not specified, project name will be used..
The default is None.
- **dd_strict_mode** = *boolean*
Enable or disable creation of code that is not loadable.
The default is False.
- **dd_tester_model** = *string*
'P1000', 'P800', 'P600', 'C400e' and 'C200e' are valid models for 93000SD. 'PS400', 'PS800' and 'PS3600' are valid models for 93000DD. 'PS9G' and 'PS1600' are valid models for 93000SS.
The default is 'P800' for 93000SD, 'PS800' for 93000DD, 'PS1600' for 93000SS..
- **dd_testmethod_class** = *string*
Testmethod class, used in UTM and classic modes. The default suits only UTM mode, please replace it with 'functional' for classic mode.
The default is 'ac_tml.AcTest.FunctionalTest'.
- **dd_testmethod_limits** = *string*
Testmethod limits, used in UTM and classic modes.
The default is "".
- **dd_testmethod_mode** = *string*
Testmethod mode, may be 'utm', 'classic' or 'functions'.
The default is 'functions'.
- **dd_testmethod_name** = *string*
Testmethod name, used in classic mode only.
The default is "".
- **dd_testmethod_params** = *string*
Testmethod parameters, used in UTM and classic modes.
The default is "".
- **dd_tim_comments** = *boolean*
Add STIL information as comments to timing files.
The default is True.
- **dd_tim_group_exclude_pins** = *array*
Exclude pins from grouping in timing files.
The default is [].
- **dd_use_channel_mapping** = *string*
Use specific channel mapping from STIL Environment block.
The default is None.

- **dd_use_eqfs** = *boolean*
Use EQFS commands in timing and level files.
The default is False.
- **dd_use_mhz_specs** = *boolean*
Enable or disable the use of MHz frequency in AC specs.
The default is False.
- **dd_use_model_file** = *string*
Check tester model limitations based on the values read from a 93000 model file.
The default is None.
- **dd_use_retags** = *boolean*
Enable or disable the use of STIL resource tags to generate device cycle names.
The default is True.
- **dd_use_tim_groups** = *boolean*
Enable or disable group definitions in timing files.
The default is True.
- **dd_vector_memory_summary** = *boolean*
Print vector usage summary for each binary pattern.
The default is False.
- **dd_wavetbl_remapping** = *dictionary*
Allows you to rename device wavetable names by means of user-supplied table.
The keys are <WFT1>_<WFT2>... strings (WFTs are sorted) for regular setups
and <WFT1>_<WFT2>...<PORT>for multiport setups. The values are used-
defined wavetable names..
The default is {}.

Example:

```
1 dd_wavetbl_remapping = { 'default_load_unload' : 'WVT1' }
```

- **dd_xmode_aware_comments** = *boolean*
If all comments for all sub-X-Mode vectors are identical generate only single
comment,when each comment seperated by '|'.
The default is False.
- **dd_xmode_based_statemapping** = *boolean*
Statemapping is based on selected x-mode.
The default is True.
- **dd_xmode_factor** = *boolean*
Apply X-Mode factor to all patterns.
The default is True.
- **dd_xmode_all_cmb** = *boolean*
Enable or disable all possible X-Mode combinations in timing instead of only
actual timing combinations in the patterns.
The default is False.
- **dd_yield_learning** = *boolean*
Yield learning (Nautilus) support.
The default is False.

- **dd_yield_learning_scan_domain = string**
Yield learning - scan structures domain name used in protocol file. 93000DD/93000SS targets only..
The default is None.
- **dd_yield_learning_scan_structures = string**
Yield learning - scan structures selection.
The default is None.
- **dd_yield_learning_skip_flops = boolean**
Yield learning - skip flops.
The default is False.

Chapter 5

Generating Advantest 93000 (SmarTest 8.X) Output From STIL

5.1 How Do I Generate Advantest 93000 (SmarTest 8) Output from STIL?

```
> ategen -target 93000SMT8 mysource.stil
```

For more information see ?? in Section ?? and ?? in Section ??.

5.2 Types of Generated Files and File Naming Convention

The table below shows 93000 (SmarTest 8.X) files generated by VCD93K with a short description. Output files are created in working directory (further denoted as *[workdir]*, see Section ??) and use project directory (further denoted as *[project]*, see Section ??) in their names.

File Type	File Name
<i>[workdir]</i> / <i>[project]</i> .prog	project file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /common/ <i>[project]</i> .dbd	channels file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /common/ <i>[project]</i> _groups.spec	groups file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /common/ <i>[project]</i> .flow	flow file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /func/ <i>[project]</i> _tim.spec	timing sets file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /func/ <i>[project]</i> _wvt.spec	wavetables file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /func/ <i>[project]</i> _lev.spec	DC levels file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /func/ <i>[project]</i> _specs.spec	specs file
<i>[workdir]</i> /src <i>[project]</i> / <i>[project]</i> /func/*.pat	pattern files

Table 5.1: Advantest 93000 (SmarTest 8.X) File Naming

5.3 STIL Elements Usage

5.3.1 Signal and SignalGroup Translation

The information from STIL Signals block is used to create the 93000 (SmarTest 8.X) channels (.dbd) file. STIL signal names are used as the pin names in the target test program.

STIL Signals block example:

```
1 Signals {  
2     clk In;  
3     data InOut;  
4     Vcc Supply;  
5 }
```

Corresponding 93000 (SmarTest 8.X) channel definitions:

```
1 dutboard my_project {  
2     sites = 1;  
3     signal clk {  
4         site 1 { pogo=10101; }  
5     }  
6     signal data {  
7         site 1 { pogo=10102; }  
8     }  
9     signal Vcc {  
10        site 1 { pogo=11; }  
11    }  
12 }
```

Channel allocation can be defined by using the optional STIL Environment block. The NameMaps that define channels must start with 93000Channel followed by semicolon and configuration name (e.g. 93000Channel:Standard).

STIL Environment block example:

```
1 Environment {  
2     NameMaps "93kChannel:Standard" {  
3         Signals {  
4             clk "10201";  
5             data "10303";  
6         }  
7     }  
8 }
```

Corresponding 93000 (SmarTest 8.X) channel definitions:

```
1 dutboard my_project {
2     sites = 1;
3     signal clk {
4         site 1 { pogo=10201; }
5     }
6     signal data {
7         site 1 { pogo=10303; }
8     }
9 }
```

STIL SignalGroups block is used to create the 93000 (SmarTest 8.X) groups file.

STIL SignalGroups block example:

```
1 SignalGroups {
2     AllS = 'clk + data';
3 }
```

Corresponding 93000 (SmarTest 8.X) group definitions:

```
1 spec my_project_groups {
2     group AllS = clk + data;
3 }
```

5.3.2 Timing Translation

The STIL Timing block selected in the active PatternExec block is used to generate 93000 (SmarTest 8.X) timing. Multiple WaveformTable blocks are merged into one 93000 wavetable, but they must share the same Period. Each STIL Waveform is translated into a 93000 waveform. Edge allocation is performed automatically.

VCD93K creates a timing sets file (..._tim.spec) that defines timing sets and edges; and wavetables file (..._wvt.spec) that defines wavetable and waveforms.

STIL Timing block example:

```
1 Timing {
2     WaveformTable t1 {
3         Period '100ns';
4         Waveforms {
5             clk { P { '0ns' D; '25ns' U; '75ns' D; } }
6             data { 01 { '50ns' D/U; } }
7         }
8     }
9 }
```

93000 (SmarTest 8.X) timing sets file:

```
1 spec my_project_tim {
2     setup digInOut clk {
3         set timing t1 {
4             period = 100.0 ns;
5             d1 = 0.0 ns;
6             d2 = 25.0 ns;
7             d3 = 75.0 ns;
8         }
9     }
10    setup digInOut data {
11        set timing t1 {
12            period = 100.0 ns;
13            d1 = 50.0 ns;
14        }
15    }
16 }
```

93000 (SmarTest 8.X) wavetables file:

```
1 spec my_project_wvt {
2     setup digInOut clk {
3         wavetable t1 {
4             xModes=1;
5             P : d1:0 d2:1 d3:0;
6         }
7     }
8     setup digInOut data {
9         wavetable t1 {
10            xModes=1;
11            0 : d1:0;
12            1 : d1:1;
13        }
14    }
15 }
```

93000 Wavetable Naming

For non-multiport setups each wavetable is assigned a name based on all STIL WFTs that are merged into this wavetable. For multiport setups, a wavetable is created for each port and named as above with an addition of the port name. All wavetable names are trimmed to fit SmarTest maximum wavetable name limit.

Timing Events Mapping

STIL states are translated to 93000 (SmarTest 8.X) edges using the following table. Duplicated drive states (e.g. D followed by another D) are removed.

STIL Event	STIL Symbol	93000 (SmarTest 8.X) Edge
Logic Low	A	F0N
Logic High	B	F1N
ForceDown	D	F00
ForceUp	U	F10
ForceOff	Z	FNZ
ForcePrior	P	FN0
ForceUnknown	N	FN0
CompareLow	L	L
CompareHigh	H	H
CompareValid	V	X
CompareOff	T	I
CompareLowWindow	l	WL
CompareHighWindow	h	WH
CompareOffWindow	t	WI
CompareValidWindow	v	WX
CompareUnknown	X, x	WC, X
LogicZ	F	FNZ
ExpectLow	R	X
ExpectHigh	G	X
ExpectOff	Q	skipped
Capture	C	C

Table 5.2: Mapping Between STIL Events and 93000 (SmarTest 8.X) Edges

5.3.3 DCLevels Translation

The STIL DCLevels block selected in the active PatternExec block is used to generate 93000 (SmarTest 8.X) levels file (..._lev.spec). Name of the DCLevels block used as 93000 (SmarTest 8.X) level set.

The STIL DCLevels are mapped to 93000 levels using the following table:

STIL DCLevels	93000 (SmarTest 8.X) Pin Levels
VIH	vih
VIL	vil
VOH	voh
VOL	vol

Table 5.3: STIL DCLevels to 93000 (SmarTest 8.X) Pin Levels

When STIL DCLevels are not specified, the VCD93K generates default levels as follows:

93000 (SmarTest 8.X) Pin Level	Default Value
vih	3.3V
vil	0V
voh	2.4V
vol	0.8V

Table 5.4: Default 93000 (SmarTest 8.X) Pin Levels

Example of STIL DCLevels block:

```
1 DCLevels DC1 {  
2   clk      { VIH '4.8V'; VIL '0V'; }  
3   data     { VIL '0V'; VIH '4.8V'; VOL '0V'; VOH '5V'; }  
4 }
```

Corresponding 93000 (SmarTest 8.X) levels file:

```
1 spec project_lev {
2     setup digInOut clk {
3         set level DC1 {
4             vih          = 4.8 V;
5             vil          = 0.0 V;
6         }
7     }
8     setup digInOut data {
9         set level DC1 {
10            vih          = 4.8 V;
11            vil          = 0.0 V;
12            voh          = 5.0 V;
13            vol          = 0.0 V;
14        }
15    }
16 }
```

5.3.4 Specs Translation

STIL Spec and Selector blocks are used to create the 93000 (SmarTest 8.X) specs file.

Example of STIL Spec and Selector blocks:

```
1 Spec S1 {
2     Category C1 {
3         Perl    { Min '100ns';   Max '200ns';   Typ '150ns'; }
4         Tpd     { Min '10ns';    Max '20ns';    Typ '15ns';  }
5         vcc     { Min '4.5V';    Max '5.5V';    Typ '5.0V';  }
6     }
7 }
8 Selector S {
9     Perl Typ;
10    Tpd  Min;
11    vcc  Min;
12 }
```

Corresponding 93000 (SmarTest 8.X) specs file:

```
1 spec my_project_specs {
2     Perl    = 150.000000 ns;
3     Tpd     = 10.000000 ns;
4     vcc     = 4.500000 V;
5 }
```

5.3.5 Pattern Translation

Pattern blocks in STIL have a one-to-one relationship to 93000 (SmarTest 8.X) pattern files (*.pat). Exceptions to this rule occur when a pattern is used across several ports. All pattern files are located in the "src[project]/[project]/func" directory; each pattern is named based on the STIL Pattern block name.

93000 (SmarTest 8.X) patterns are ASCII files in XML format. Each pattern consists of Instrument section (specifying used signals and pattern instructions) and Vectors section (containing pattern vectors).

STIL Elements Usage Generating Advantest 93000 (SmarTest 8.X) Output From STIL

The following example shows an input STIL pattern and a 93000 (SmarTest 8.X) pattern as generated by VCD93K.

```
1 Pattern P1 {
2   W t1;
3   V { clk = P; data = 0; }
4   Loop 100 {
5     V { clk = P; data = 1; }
6   }
7   V { clk = P; data = 0; }
8 }
```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <Pattern xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:noNamespaceSchemaLocation="Pattern.xsd">
4     <Program>
5         <Instrument id="clk,data">
6             <Instruction id="genVec" value="1"/>
7             <Instruction id="loop" value="100"/>
8             <Instruction id="genVec" value="1"/>
9             <Instruction id="loopEnd"/>
10            <Instruction id="genVec" value="1"/>
11        </Instrument>
12    </Program>
13 <Vector>
14 <![CDATA[
15 P0
16 P1
17 P0
18 ]]>
19 </Vector>
20 </Pattern>
```

STIL PatternExec and PatternBurst blocks are used to generate 93000 (SmarTest 8.X) test flow file.

Example:

```
1 PatternBurst B1 {
2   PatList {
3     P1;
4     P2;
5     P3;
6   }
7 }
8 PatternExec PE1 {
9   PatternBurst B1;
10 }
```

Corresponding 93000 (SmarTest 8.X) test flow file:

```
1 flow project {
2     setup {
3         suite multipat calls actml.FunctionalTest {
4             measurement.specificationName = setupRef(project.func.project_specs);
5             measurement.patternName = setupRef(project.func.P2,project.func.P3,
6                 project.func.P1);
7         }
8     }
9     execute {
10         multipat.execute();
11     }
12 }
```

5.3.6 Using MultiPort

It is possible to split STIL pattern into multiple 93000 (SmarTest 8.X) patterns, each for a separate group of pins (called port). Ports can be specified using optional STIL block Environments. Name of port specified in the Environment is used as suffix in 93000 (SmarTest 8.X) pattern names, timing sets names and wavetable names.

For example, the below definitions will lead to generation of two 93000 (SmarTest 8.X) patterns (P1_Clock and P1_Data), two timing sets and two wavetables (t1_Clock and t1_Data).

```
1 STIL 1.0 { Design D22; }
2 ...
3 SignalGroups {
4     cg = 'clk';
5     dg = 'data';
6 }
7 Environment {
8     NameMaps "93kPorts" {
9         SignalGroups {
10             cg "Clock";
11             dg "Data";
12         }
13     }
14 }
15 Timing {
16     WaveformTable t1 {
17         Period 'tc';
18         Waveforms {
19             clk      { P { 'trise' U; 'tfall' D; } }
20             data      { 01 { 'trise-tsetup' D/U; } }
21         }
22     }
23 }
24 ...
25 Pattern P1 {
26     W t1;
27     V { clk = P; data = 0; }
28 }
```

5.4 Elements Naming

Element name	93000 elements
timing wavetable (No MultiPort)	[stil_wft_1]_[stil_wft_2]...[stil_wft_n]
timing wavetable (MultiPort)	[stil_wft_1]_[stil_wft_2]...[stil_wft_n]_[port]
timing set	same as wavetable
level set	[stil_level_name]
test_suite_name	[stil_pattern]
pattern	[stil_pattern]

Table 5.5: Elements Naming

5.5 Customizing 93000 Files

The customization of the 93000SMT files is done using the VCD93K configuration file. Following is an example of a configuration file:

```

1 smt8_smarTest_version = "8.0.0"
2 smt8_expand_loops      = 1
3 smt8_use_tim_groups    = 0
4 smt8_skip_unused_signal_groups = 1

```

The table below describes the complete set of 93000 (SmarTest 8.X) package options.

- **smt8_allow_nonaligned_match_loops = *boolean***
Allow match loops with number of vectors not aligned to x-mode.
The default is 0.
- **smt8_auto_create_signal_groups = *boolean***
Create all inputs and all outputs signal groups for debug purposes.
The default is 0.
- **smt8_combinations_allow_new_waveforms = *boolean***
Allow addition of new waveforms in combinations file.
The default is 1.
- **smt8_combinations_file = *string***
Turns on waveform incremental mode when set to a file path. The combinations file allows generation of multiple STIL patterns, while keeping consistent timing, compatible with all generated binary patterns. This mode requires identical timing to be shared between all STIL patterns. It is most useful in case of X-Modes, when different STIL patterns have different combinations of STIL waveforms. Waveform incremental mode is compatible with multiport..
The default is none.
- **smt8_comment_max_size = *integer***
Data size for a single BCMT vector comment command.
The default is 640000.
- **smt8_create_min_max_typ_tests = *boolean***
Creates a separate SPECSET for Typ, Min and Max AC and DC spec values. This allows to run the same test with Typ, Min or Max selectors.
The default is 0.

- **smt8_dcap_vector_variable_rename** = *boolean*
Rename default vector variable in case of digital capture, The entry created from the name of DCapFilter block + _ + pattern name..
The default is none.
- **smt8_digital_capture_mode** = *string*
Choose SmarTest capture mode between 'std_selective' or 'HRPF'..
The default is 'std_selective'.
- **smt8_digital_capture_res** = *string*
Choose smartest capture resolution between 'std' or 'double'..
The default is 'std'.
- **smt8_error_on_long_comments** = *boolean*
Error out on comments violating SmarTest limit (depends on SmarTest version).
The default is 0.
- **smt8_expand_loops** = *boolean*
Expand original loops in source STIL files.
The default is 0.
- **smt8_full_incremental_log** = *boolean*
Enable or disable printing full incremental mode report.
The default is 0.
- **smt8_generate_hvm** = *boolean*
HVM support. Generates VCAT pattern command..
The default is 0.
- **smt8_generate_vmap** = *boolean*
Enable or disable scan diagnostics.
The default is 0.
- **smt8_gzip_pattern** = *boolean*
Enable or disable GZip compression of vector files.
The default is 0.
- **smt8_hvm_cycle_info** = *boolean*
HVM vector and cycle information..
The default is 0.
- **smt8_levels_group_exclude_pins** = *array*
Exclude pins from grouping in level files.
The default is none.
- **smt8_optimize_receive_edges, smt8_optimize_drive_edges** = *boolean*
Enable or disable edge optimization.
The default is 1.
- **smt8_pad_xmode_vectors** = *boolean*
Pad last x-mode vector .
The default is 0.
- **smt8_pin_based_tim** = *boolean*
Enable or disable pin-based timing.
The default is 0.

- **smt8_repeat_break** = *integer*
Loops will be flattened if loop count is less than repeat break value. Loops will be flattened by default if $(\text{loop_size} * \text{loop_count} + 6)/7 < 1 + (\text{loop_count} + 6)/7$.
The default is none.
- **smt8_repeat_compression** = *boolean*
Compress identical vectors to single vector loops.
The default is 0.
- **smt8_repeat_compression_ignore_comments** = *boolean*
When repeat compression is on: compress identical vectors to single vector loops although they don't have the same comment. The different comments from the vectors will be placed before the created loop..
The default is 1.
- **smt8_skip_unused_signal_groups** = *boolean*
Don't create signal group definitions if those groups are not used in timing and level declarations in STIL.
The default is 0.
- **smt8_smarTest_version** = *string*
Controls output behavior depending on SmarTest version.
The default is '8.0.0'.
- **smt8_strict_mode** = *boolean*
Enable or disable creation of code that is not loadable.
The default is 0.
- **smt8_tester_model** = *string*
'PS1600' is the only valid model..
The default is 'PS1600'.
- **smt8_testmethod_class** = *string*
Testmethod class, used in UTM and classic modes. The default suits only UTM mode, please replace it with 'functional' for classic mode.
The default is 'ac_tml.AcTest.FunctionalTest'.
- **smt8_testmethod_limits** = *string*
Testmethod limits, used in UTM and classic modes.
The default is "".
- **smt8_testmethod_mode** = *string*
Testmethod mode, may be 'utm', 'classic' or 'functions'.
The default is 'functions'.
- **smt8_testmethod_name** = *string*
Testmethod name, used in classic mode only.
The default is "".
- **smt8_testmethod_params** = *string*
Testmethod parameters, used in UTM and classic modes.
The default is "".
- **smt8_tim_comments** = *boolean*
Add STIL information as comments to timing files.
The default is 0.

- **smt8_tim_group_exclude_pins** = *array*
Exclude pins from grouping in timing files.
The default is none.
- **smt8_use_channel_mapping** = *string*
Use specific channel mapping from STIL Environment block.
The default is none.
- **smt8_use_tim_groups** = *boolean*
Enable or disable group definitions in timing files.
The default is 1.
- **smt8_vector_memory_summary** = *boolean*
Print vector usage summary for each binary pattern.
The default is 0.
- **smt8_xmode_aware_comments** = *boolean*
If all comments for all sub-X-Mode vectors are identical generate only single comment,when each comment seperated by '|'.
The default is 0.
- **smt8_xmode_factor** = *boolean*
Apply X-Mode factor to all patterns.
The default is 1.
- **smt8_xmode_all_cmb** = *boolean*
Enable or disable all possible X-Mode combinations in timing instead of only actual timing combinations in the patterns.
The default is 0.
- **smt8_yield_learning** = *boolean*
Yield learning (Nautilus) support.
The default is 0.
- **smt8_yield_learning_scan_domain** = *string*
Yield learning - scan structures domain name used in protocol file..
The default is none.
- **smt8_yield_learning_scan_structures** = *string*
Yield learning - scan structures selection.
The default is none.
- **smt8_yield_learning_skip_flops** = *boolean*
Yield learning - skip flops.
The default is 0.

Chapter 6

Appendix

6.1 What is CSV File ?

The Comma Separated Value (CSV) file format is actually a text file format which can be edited as excel (open office) file.

6.2 How to use CSV File?

Using CSV file provides an alternative way to define include signals and their attributes, such as type, direction controls and input/output waveforms timing.

In order to generate CSV template is using `csv_export_pins` command line parameter:

```
> vcd93k -csv_export_pins mycsv.csv myvcd.vcd
```

To import signals data from CSV, use setup file like the following one:

```
input_files = [ 'vcd1.vcd' ]

# definitions of start time, cycle width etc.

csv_import_pins( 'vcd1.csv', delimiter = ',' )
```

Below is an example of CSV file before and after editing it. In the CSV file we can define:

- The direction of the pins (input,output,inout)
- IO control for inout signals in VCD file.
- input/output waveform timing for the signals

	A	B	C	D	E
1	VS Pin Attributes	1			
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform
3	##HELP##	in/out/inout	in:pin=states,out:pin=states	time:state ... time:state <u>sp</u> =time	time:state ... time:state <u>sp</u> =time
4	D_djr	in			
5	A0	in			
6	A1	in			
7	CLK0	in			
8	CLK1	in			
9	CLK2	in			
10	CS	in			
11	D0	in			
12	D1	in			
13	D2	in			
14	D3	in			
15	D4	in			
16	D5	in			
17	D6	in			
18	D7	in			
19	GATE0	in			
20	GATE1	in			
21	GATE2	in			
22	OUT0	in			
23	OUT1	in			
24	OUT2	in			
25	RD	in			
26	WR	in			

Figure 6.1: CSV file before editing

Example of the output of CSV file from VCD file, as you can see all the pins configure as input for VCD file.
for EVCD file inout is the default direction.

	A	B	C	D	E
1	VS Pin Attributes	1			
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform
3	##HELP##	in/out/inout	in:pin=states	out:pin=states	time:state ... time:state sp=time
4	A0	in		150ns:S	
5	A1	in		150ns:S	
6	CLK0	in		0ns:D 200ns:U	
7	CLK1	in		0ns:D 200ns:U	
8	CLK2	in		0ns:D 200ns:U	
9	CS_	in		150ns:S	
10	D0	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
11	D1	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
12	D2	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
13	D3	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
14	D4	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
15	D5	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
16	D6	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
17	D7	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
18	GATE0	in		150ns:S	
19	GATE1	in		150ns:S	
20	GATE2	in		150ns:S	
21	OUT0	out			250ns:S
22	OUT1	out			250ns:S
23	OUT2	out			250ns:S
24	RD_	in		150ns:S	
25	WR_	in		150ns:S	

Figure 6.2: the CSV file after editing

As you can see We edited the CSV file as follows:

1. Edit Pin direction:

- I. We configure the D0..D7 signals to be inout and IO control for D0..D7 pins depending on D_dir pin.
- II. OUT0..OUT2 to be Output

2. Define the timing for the pins (the cycle width here is 400ns)

For input signals we define '100ns:D 200ns:U' it means that every 400ns at 100ns the signal drive low and at 200ns it drives high.
the signal will look like:

'100ns:D 200ns:U 500ns:D 600ns:U 900ns:D 1000ns:U'

For output signals we define '100ns:S sp=50ns' it means that every 400ns at 50ns the signal will be sampled and the sample will be put at 100ns.

For example if in the (E)VCD file we have output signal which look like: '20ns:L 200ns:H 440ns:L 750ns:H 900ns:L 1050ns:Z'

the output that will be generated: '100ns:L 500ns:Z 900ns:H

(we sampled the values at 50ns, 450ns and 850ns...)

6.3 How to define pins direction ?

In VCD format, same states (0, 1, X, Z) are used to designate input and output events. Therefore for bidirectional signals, a mechanism is needed to correctly translate VCD states to input or output states. In VCD93K translation of inout signals in VCD files can be determined by use of direction control, as shown in the setup file below.

```
input_files = [ 'pool/ex2.vcd' ]
include_pins( pinlist = [ 'D?', 'Dio' ] )
cycle_width = "100ps"

pin_attributes(                # input signal
    pinlist                    = ['Di'], # specify signal name
    type                       = 'in'   # specify type
)

pin_attributes(                # output signal
    pinlist                    = ['Do'], # specify signal name
    type                       = 'out'   # specify type
)

pin_attributes(                # inout signal - use direction control
    pinlist                    = ['Dio'], # specify signal name
    type                       = 'inout', # specify type
    io_control                  = 'in:C_in=1,out:C_in=0', # specify direction control
)
```

6.4 How to define input/output waveforms timing ?

In VCD93K it is mandatory to specify predefined (or "forced") waveforms for all the signals, using "pin_attributes" setup file command or CSV file.

Two timing waveforms can be defined: one for input cycles and one for output cycles. VCD93K selects one of these waveforms based on a signal value in the (E)VCD file. See below for explanation where exactly the signal is sampled.

The following states can be used in the timing waveforms:

- D: Driver on, drive low.
- U: Driver on, drive high.
- Z: Driver off.
- X: Don't sample (mask).
- S: Use pattern data.

In addition to a subset of standard STIL states (intermediate format), predefined waveform can contain a special state: "use pattern data" (S). This state is translated to different regular STIL states depending on actual value of the signal in the (E)VCD source file. By default, the signal value is sampled in the "S" events position; "sp" parameter can be used to force different sample point.

Value of the signal in the (E)VCD source file at the same time point is also used to determine which of the forced waveform (input or output) will be used by VCD93K . In absence of "S" event and "sp" parameter then not data will be sampled.

For example look in the example of how to use CSV file section 4.1

6.5 How to use FSDB dump files

The VCD93K does not directly support files in FSDB format, such files must be converted first into VCD format. FSDB to VCD converter is available from [SpringSoft](#).

Chapter 7

Glossary

STIL	<i>Standard Test Interface Language</i> , developed by IEEE for the design-to-test flow of the semiconductor test industry.
VCD	<i>Value Change Dump</i> , format for representing DUT simulation trace data. Uses 4-state representation of variable changes (1,0,X,Z) with no strength information.
EVCD	<i>Extended Value Change Dump</i> , VCD extended for representing variable changes in more states and with strength information.
ATE	<i>Automatic Test Equipment</i> , used for quality check of each manufactured integrated circuit with compliance to the matching test program.
DUT	<i>Device Under Test</i> , the device model or the actual integrated circuit.
IEEE	<i>Institute of Electrical and Electronics Engineers, Inc</i> , continues to develop sophisticated yet complex modules to <i>STIL</i> .